

Versal ACAP AI Engine Heavy-Ion Testing with Spill Synchronization

Garrett Smith, Caleb Price, Brigham Morgan, Ted Wilcox, Greg Allen, and Michael Wirthlin

ABSTRACT

The AMD Versal is a complex heterogeneous architecture that includes programmable logic, multicore processors, and a large array of Artificial Intelligence (AI) engines. The AI engines are of interest in space applications because they support high-throughput AI inference more efficiently than traditional processors. This paper describes a methodology for heavy-ion single-event effect (SEE) testing of all 400 AI engine tiles of the XCVC1902 device simultaneously at the NASA Space Radiation Laboratory (NSRL) facility. The testing approach extracts all AI engine data after each spill of the beam structure to identify the failure modes of each AI engine on a spill-by-spill basis. Heavy-ion cross-section estimates obtained from this testing approach are presented for AIE freezing, AIE memory bit upsets, AIE calculation errors, and APU halting.

I. INTRODUCTION

There is growing interest in performing real-time AI inference in space environments [1], [2]. This interest is driven by the availability of high bandwidth sensors, increased onboard data generation, and the need for real-time decision making in remote environments. Performing AI inference in an embedded space environment, however, is challenging due to the high power requirements of traditional GPU architectures and the harsh radiation environment in space [3], [4].

There is significant interest in using commercial off-the-shelf (COTs) embedded devices that are designed for low-power, high-performance AI inference in a space environment. These devices are designed to perform AI inference more efficiently by using specialized hardware architectures such as customized AI engines, custom logic/instructions, and programmable logic. Unlike GPUs that perform AI inference using floating-point operations, these devices use specialized cores to perform inference

using fixed-point operations. The 7 nm FinFET AMD Versal device is an example of such a device [5].

Accelerated radiation testing is performed on such devices to understand the impact of radiation on real-time AI inference workloads and to estimate the failure rates in space environments. While these experiments are helpful for understanding the overall failure rate of the application of interest, it is difficult to generalize the results from such tests to other applications or AI models. Rather than measuring the behavior of a single AI inference application, this work seeks to measure the failure behavior of individual AI engines. If the failure behavior of individual AI engines is known, it may be possible to estimate the failure behavior of a wide variety of AI applications that use an arbitrary number of AI engines.

This paper introduces a unique testing procedure that performs individual AI engine tests during each “spill” of the NSRL beam structure. The operating state and memory of all 400 of the AI engines were extracted after each spill to identify spill-specific failure events. A dedicated AI engine tests were created to test each of the following structures of the AI engine: the scalar unit, fixed vector unit, floating vector unit, data transfer unit and the program and data memory.

The contributions of this paper are as follows. First, this paper demonstrates a novel approach for testing many processors simultaneously by exploiting the beam spill structure at NSRL. To our knowledge there are no previous works that have tested hundreds of processing cores simultaneously while also isolating the failure behavior of each core. Second, the results from these tests provide insight into the failure modes of these AI engines and the corresponding cross-section at multiple LET values. Third, the relative cross-section of the AI engines is compared to other architectural elements such as internal memories, the application processors, and platform management infrastructure. With component-level failure data, it is possible to estimate the failure rates of a wide variety of applications that use an arbitrary number of AI engines.

II. RELATED WORK

There is great interest in understanding the reliability of AI inference in space environments, especially with commercially available embedded solutions [6]. Many studies evaluate the reliability of AI inference applications in radiation environments for a variety embedded architectures including Processor/FPGA hybrid devices [7]–[9],

Corresponding author email: wirthlin@byu.edu

Garrett Smith, Caleb Price, Brigham Morgan and Michael Wirthlin are with the Dept. of Electrical and Computer Engineering, Brigham Young University, Provo, Utah 84602.

Ted Wilcox is with the NASA Goddard Space Flight Center, Greenbelt, Maryland 20771.

Greg Allen is with the Jet Propulsion Laboratory at the California Institute of Technology, Pasadena, California 91109.

Heavy-Ion Beam testing was provided by the DECPTR program.

Funding was provided by the Center for Space, High-Performance and Resilient Computing (SHREC), <https://www.nsf-shrec.org/>.

embedded GPUs [10], [11], multicore processors [12], and dedicated AI accelerators [13]. In these examples, an AI inference application is tested with radiation to obtain an overall failure rate or cross-section of a specific application on a specific device. In this work, we seek to identify the failure modes and rates of individual internal AI engine components rather than full AI applications in order to estimate the failure mode of arbitrary AI applications based on the number of AI engines used.

Previous works have performed radiation test on the same Versal device used in this work including cross-section measurements of multiple subcomponents using heavy-ion and proton testing [14]. Maillard et al. tested the processing system and reported the overall processing system SEFI FIT at sea level [15]. Quinn et al. performed a neutron test evaluating the SEFI and SEU behavior of the APU and RPU processors under different benchmarks [16]. Most relevant to this work, Maillard et al. tested the AI engines under proton beam [17] in which the reliability of a full machine learning application using the AIE Deep Learning Processing Unit was measured. Unlike that work, this paper isolates individual AI engine components to provide cross-section data that can be generalized to a wider variety of applications.

Quinn reviewed best practices for component-level radiation testing of complex devices, noting that isolating failures to specific components within a heterogeneous architecture is particularly difficult [18]. In a study of the High-Performance Space Computer (HPSC), the authors highlight the difficulty of testing and isolating multiple subsystems for radiation effects [19]. A proton radiation test of a GPU was conducted [20] with an emphasis on testing individual components within the system. Michalak et al. tested processors from the Roadrunner supercomputer under neutron beam and found that faults in individual processors could cause system crashes or silent data corruption across the entire platform [21]. An approach for testing multiple heterogeneous components of an SoC was demonstrated [22], but this was limited to a few of cores and I/O devices.

There are few examples of radiation testing that seeks to identify failures in individual cores within many core systems. The Maestro 49-core RHBD processor was tested with heavy ions to measure single-event effects of the caches and a JPEG application [23]. Vargas et al. demonstrated that hundreds of cores can be irradiated simultaneously by running a parallel matrix multiplication application on a 256-core processor [24]. In both cases the radiation testing was performed at the application level to identify overall system failures rather than isolating the failure behavior of individual cores. In this work, a heavy-ion testing approach was developed to measure the failure behavior of each of the 400 AI engines on the Versal device individually and simultaneously.

III. VERSAL OVERVIEW

The Versal ACAP device is a heterogeneous architecture that includes programmable logic, multicore processors, DDR and PCIe controllers, and a large array of AI engines to perform machine learning operations. Although the focus of this paper is on the AI/ML engines, essential components needed to support the AI engines include the platform management controller (PMC), the application processing unit (APU), and the network on chip (NoC).

A. Platform Management Controller (PMC)

The platform management controller (PMC) handles the configuration and booting of the device, environmental monitoring, and configuration memory scrubbing and is critical to the operation of the entire device. The PMC contains two triplicated processors to manage the system platform. Custom firmware was created for the test to monitor several SEE induced events including PL configuration memory (CRAM) upsets, PMC memory upsets, TMR mismatches, and system exceptions.

B. Application Processing Unit (APU)

The application processing unit (APU) is a dual-core ARM Cortex-A72 and is used to support the operation of the AI engines. The first APU core (APU0) was used to communication with and control the AI engines while the second APU core (APU1) was running a simple benchmark. Whenever the APU0 was not managing the AI engines, it ran the same benchmarks as APU1. Failures in the APU will prevent the AI engines from operating correctly as the APU is responsible for starting the AI engines.

C. Other Versal Components

Other components tested simultaneously with the AI engines include the real-time processing unit (RPU), the network on chip (NoC), the PCIe controller, and the programmable logic (PL). Failures in these components were logged but are not reported in this paper.

IV. AI ENGINES

The primary component of interest for this paper is the AI tile. AI tiles are arranged in a 2D array-like structure, each composed of an AI engine, a memory module, and an interconnect. Each AI engine has a scalar unit along with fixed-point and floating-point vector units [5]. These units along with their associated load/store units are shown in Figure 1. The AI cores are considered very long instruction word (VLIW) processors that perform multiple operations in parallel including a scalar operation, two data moves, two vector reads, one vector write, and one vector instruction [25].

Each AI engine can access four data memory modules: its own, and those to the north, south, and either east or west, depending on the row and relative placement of the AI engine and memory module. The first two banks of

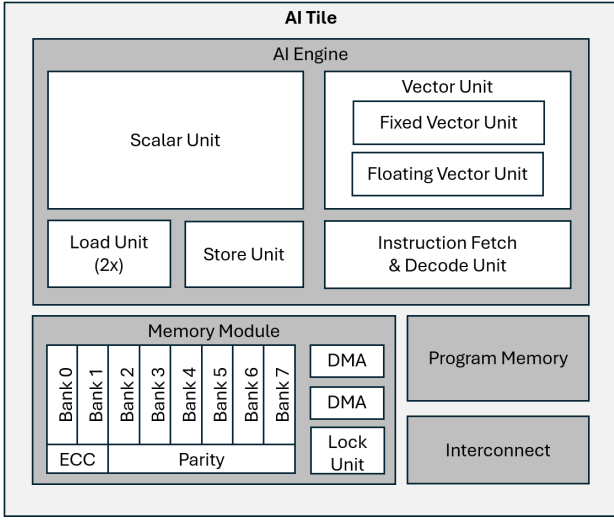


Fig. 1. AI Tile Components

each memory module are ECC protected while the other six provide parity checking. Each core also stores its own ECC-protected program memory.

A dedicated interconnect, direct memory access units, and a lock unit are used to facilitate data transfer between cores. The AI tiles are equipped with events that can be used for triggering background tasks or reporting errors that have occurred. An example is the ECC error event that is raised each time an ECC error is found in memory. During radiation testing, these events are used to collect engine failure information and memory upsets.

There are two variants of tiles, the AI engine tile on the Core series device (VCK190) and the machine learning tile (ML) on the Edge series device (VEK280). Both of these tiles were tested during the heavy-ion tests performed at NSRL.

V. AI ENGINE TEST SOFTWARE

The AI engines are designed to operate in parallel and communicate across AI tiles. For the tests described in this paper, the AI engines are isolated from each other and run independently to isolate the effects of radiation on each AI engine. Multiple tests were written to target the following units within the AI tile: scalar unit, fixed-point vector unit, floating-point vector unit, memory, and data transfer. Each test is a custom executable running on the AI engine to exercise the component of interest.

All program code and runtime data is stored locally on the cores and remains independent during the execution of the test. The cores remain idle until the APU0 core issues a start command, specifying the number of test program iterations to run. After execution, the data memory, program memory, and control registers of every AI engine are offloaded via PCIe for post-processing.

The AI Engine and memory modules contain event logic units with a defined set of local events. The control

registers have performance counters that were configured to count ECC event occurrences. These counters track SEUs during data and program memory scrubbing. Error event registers also store various error events encountered during the test. These counters are analyzed post-test to identify the cause of AI engine failure.

The data memory was used to store information during the test. Each test performed a comparison with a golden copy of data stored in ECC-protected memory. The total number of successful and failed iterations was stored in the first two banks of ECC data memory. These counts were then used when generating the calculation error cross-sections in Section VII.

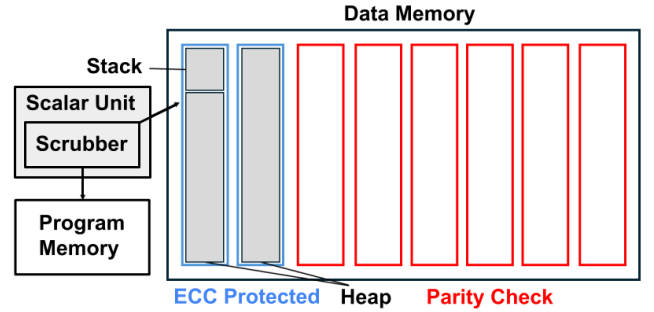


Fig. 2. AI Engine Data Memory Layout

A. Scalar Unit Test

To test the scalar unit, two 32-bit integer 21x21 constant matrices were multiplied. A square root was performed on each value in the resulting matrix (floating-point type) to include the non-linear functionality. The result was then compared against a golden value to check for any calculation errors generated during the computation. These matrices were placed in the heap within the ECC-protected memory banks (see Figure 2) to ensure that the input for the pipeline remained valid. The ECC-protected data and program memories were scrubbed to avoid non-recoverable double-bit errors.

B. Fixed and Floating Vector Unit Test

A custom test was created for the vector unit to perform matrix multiplication on the fixed-point and floating-point vector pipelines, respectively. The inputs were two 50x16 constant matrices (either integer or floating-point type) that were multiplied on the vector pipelines using vector datatypes with width 8.

C. Memory Test

The SEUs were counted for the program and data memory using the ECC architecture and memory scrubber. These counts were collected inside the performance counters. While this test ran, the scalar unit computed a matrix

multiply to keep the core performing an operation. The data for the matrix multiply was placed into the parity protected region to keep the SEUs from being overwritten before the memory scrubber could include them in the count.

D. Transfer Test

The final test measured the upset rate of transferring data between the cores. This was done by dividing the cores into pairs and establishing a ping-pong buffer in shared memory between each pair. The first kernel would fill the ping side of the buffer with all ones. Then it would increment the write value and write all twos to the pong side of the buffer. The second kernel would receive the data and validate the results against a golden value. This process was repeated during the duration of the run. Data transfers were tested in the north, south, east, and west directions.

VI. HEAVY-ION RADIATION TEST

The AI engine heavy-ion radiation tests were performed at the NASA Space Radiation Laboratory (NSRL) [26] in June and September 2024. NSRL was chosen because it provides a high energy beam that can penetrate multiple layers of dense materials to reach the active layer of the Versal device from the backside of the board. Because of the need to cool the device, the packaging material, heat sink, and cooling fan were left on the device during operation. The development board was placed in the beam with the backside of the board facing the beam path (see Figure 3) and the beam arrived at the active layer of the device from the backside. Ions with sufficient energy were used to penetrate the PCB, solder balls, and device packaging to reach the active layer. The Variable Braag Peak method was used to estimate the material depth between the PCB and the active layer [27]. This material depth estimate was then used to determine the LET values for each ion and energy.

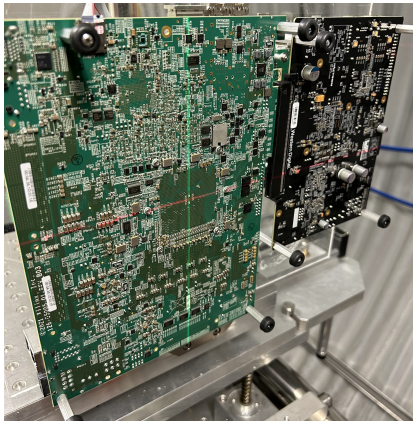


Fig. 3. Image of the backside of the VCK190 Board in NSRL Beam Path. The center of the beam is placed over the backside of the device as indicated by the laser lines.

The VCK190 and the VEK280 were used to test the two types of AI/ML engines. The setup for both boards was the same except for the number and type of AI engines within the device. The following components were also tested but not reported in this paper: the programmable logic (PL), the application and real-time processing units (APU/RPU), the platform management controller (PMC), and over a dozen internal memories.

A. Beam Spill Timing

Since the NSRL beam is driven from a synchrotron, the ions are delivered in a “spill” structure where the beam is on for a few hundred milliseconds followed by a period in which the beam is off. The dynamic behavior of the device is coordinated with the beam spill to ensure that the AI engines are active during the spill similar to the approach described in [28]. During our experiment, the beam was on for 400 ms and off for 3.8 seconds as shown in Figure 4. Each spill is treated as a dedicated “mini-experiment” in which the device is prepared for the spill, the spill is applied, and the device state is extracted after the spill but before the next spill. A beam run involves multiple spills and continues until the device hangs or too many of the AI/ML engines have failed.

To synchronize the test with the beam structure, the beam spill signal was monitored by the host computer controlling the timing of the device stimulus and data extraction. The host maintains a carefully controlled timing sequence that is synchronized with each beam spill. The AI engines are activated soon before the start of the spill to ensure that they are active during the full spill (represented by the green box) and are halted soon after the spill to allow for data extraction (represented in Figure 4 by the green box that encapsulated the beam pulse). Additional details on the spill synchronization are described in [29].

After the spill has completed, diagnostic information for all the AIE engines is extracted through the Versal PCIe interface to a local embedded PCIe platform (PolarFire Icicle kit). The timing of this data transfer is represented by the orange box. This data includes the program memory, data memory, and full register content of every AI engine. This data transfer must complete before the next spill starts to ensure that the AIE engines are running for the next spill. The data is then transferred from the embedded PCIe system to the host over Ethernet (represented by the blue box in Figure 4).

B. Test Infrastructure

The infrastructure to support the test includes a host computer, a PolarFire SoC FPGA Icicle Kit, a Versal development board (VCK190 or VEK280), the BYU JTAG Configuration Manager (JCM), a beam logger, and a power distribution unit (PDU) as shown in Figure 5.

The PolarFire SoC FPGA Icicle Kit was used for transferring data from the Versal device over PCIe. This Linux

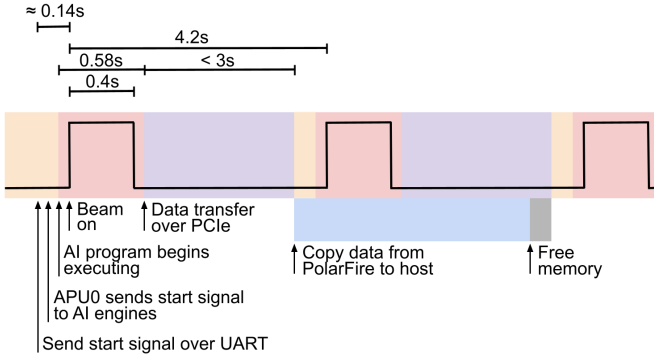


Fig. 4. Beam Structure and Data Extraction Timing

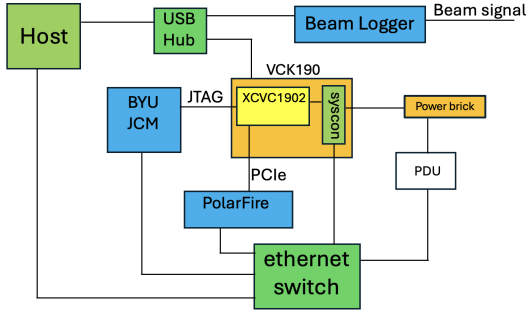


Fig. 5. Radiation Test Organization

based system was responsible for accessing the internal state of each AI engine between beam spills. The JCM device was connected to the Versal JTAG port and was responsible for extracting the five virtual UART channels through the ARM Debug Access Port (DAP). The JCM was also used to extract global memory and processor registers after each run for post-run failure analysis. A physical UART was logged to monitor the PPU processor which includes the CRAM upsets, PPU RAM upsets, and PPU status. The Versal Syscon was also logged to monitor the Versal power and temperature status.

VII. RADIATION TESTING RESULTS

Two experiments were performed at NSRL. The first occurred in June of 2024 and involved only the VCK190 board. The second occurred in September of 2024 and included both the VCK190 and VEK280 boards. The ions, their energy, and the corresponding LET used at the June and September tests are summarized in Tables I and II.

Three different types of AI engine errors were measured during the test: frozen AI engines, calculation errors, and memory errors. The cross-sections of these errors were calculated for multiple LET values and are presented in the following sections. Computing cross-sections for these tests was more complex than traditional methods, where errors are divided by fluence. In this experiment, multiple AI engines ran simultaneously, with several failing

TABLE I
JUNE 2024 IONS

Ion	Energy	LET
Fe	1000	1.2
Fe	285	2.14
Fe	190	3.21
Nb	420	4.25
Ag	460	5.42
Nb	240	6.84
Ag	260	8.65
Tb	446	10.09
Tb	290	15.26

TABLE II
SEPTEMBER TEST 2024 IONS

Ion	Energy	LET
Fe	480	1.53
Fe	190	2.93
Nb	240	6.26
Ta	310	16.42
Ta	200	28.32

after each spill. Because of this, the cross-section has to be adjusted for each spill depending on the number of active AI engines still operating during the spill. Figure 6 demonstrates the concept of AI engines dropping off during the spills for one of the runs in the June 2024 test. During the first spill, all 400 of the AI engines are active and collecting data. Five AI engines fail during this first spill, leaving only 395 AI engines active during the second spill. AI engines continue to fail throughout the run until spill 55 where only 186 AI engines remain active.

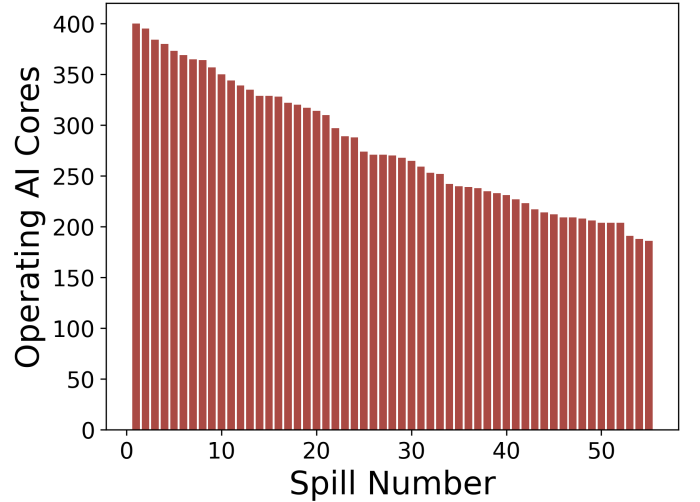


Fig. 6. Active AI Engines Per Spill for Run #27 of the June 2024 test.

Several variables are used in the cross-section equations presented below to account for spill-specific measurements and AI engine dropouts during the spills. S represents the total number of spills for a given run ($S=55$ for Run 27 described above). N represents the total number of AI engines at the start of the run where $N=400$ for the VCK190 and $N=304$ for the VEK280. The subscript i is used to denote spill-specific quantities such as Φ_i which represents the total fluence for spill i , and n_i which represents the number of AI engines that failed during spill i . The average fluence per spill for Run 27 was 2.2×10^5 p/cm^2 .

The total fluence per AI engine for a run is calculated by multiplying the fluence at each spill by the number of active AI engines during that spill and then summing across all spills, or

$$\Phi_{total} = \sum_{i=1}^S (N - n_i) \Phi_i. \quad (1)$$

This total fluence per AI engine will be used in the denominator for most of the cross-section calculations used below.

A. AIE Freeze Errors

The first type of error is a “frozen AI engine” error. This occurs when an AI engine stops responding to commands from the APU and the engine is no longer processing data. This can be determined by checking the internal registers of the AI engine after each spill. Once an AI engine is frozen, it remains unusable for the rest of the run and will no longer report any results in the remaining spills of the run.

The cross-section of frozen AI engines is calculated by dividing the total number of AI engine failures by the fluence per AI engine (see Equation 1):

$$\sigma_{frozen} = \frac{\sum_{i=1}^S n_i}{\Phi_{total}}.$$

The AI failure cross-section per for the September test of the VCK190 and VEK280 shown in Figure 7.

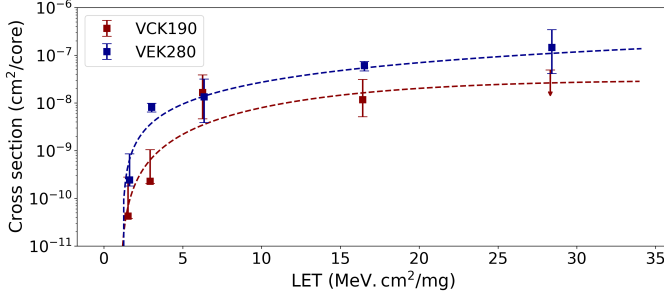


Fig. 7. AIE Freeze Failure Cross-section (September 2024 test).

B. AIE SDC Errors

The second type of error is the silent data corruption (SDC) that occurs when the AI engine produces an incorrect result. The programs running on the AI engines are self-checking and will compare the results of the computation against a golden set of results. If the results do not match, the core will increment the SDC error count stored in data memory. Four different programs were executed as described earlier to measure the difference in SDC cross-section for different processing pipelines and data transfer units.

The following equation is used to compute the SDC cross-section:

$$\sigma_{SDC} = \frac{\sum_{i=1}^S c_i}{\Phi_{total}}, \quad (2)$$

where c_i is the total number of AI engines with SDC errors counted during spill i . The SDC cross-section curves are reported for the both the VCK190 and VEK280 in Figures 8 and 9 respectively. The Weibull fit parameters for these curves are reported in Table III. It is interesting to note the variation in the results suggesting that the SDC cross-sections are heavily dependent upon the specific operation being performed by the AI engine. We recognize that the micro-benchmarks used in this work may not be representative of the actual applications running on the device.

For both devices, the relative order of the SDC cross-section for the different benchmarks was the same with the floating-point vector unit benchmark as the highest followed by the fixed-point vector unit, the scalar unit, and the data transfer unit as the lowest.

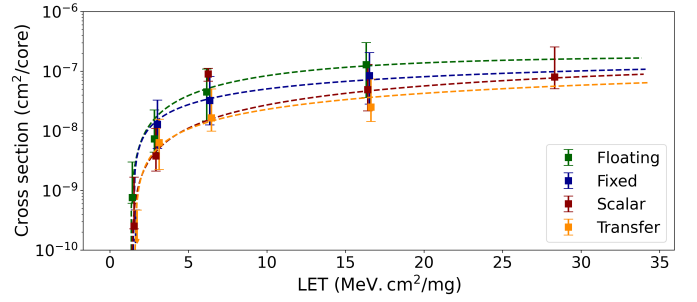


Fig. 8. AI Engine Silent Data Corruption Errors for the VCK190

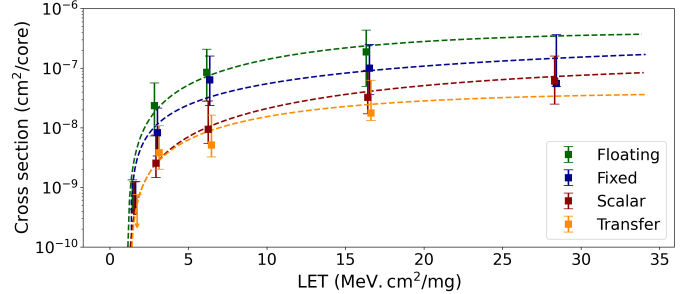


Fig. 9. AI Engine Silent Data Corruption Errors for the VEK280

TABLE III
WEIBULL FIT PARAMETERS FOR SDC BENCHMARKS

Benchmark	LET_0	W	s	σ
VCK190				
Floating	1.29	12.99	1.11	1.77×10^{-7}
Fixed	1.48	32.56	0.78	1.70×10^{-7}
Scalar	1.34	39.77	1.11	1.61×10^{-7}
Transfer	1.58	65.86	0.89	1.55×10^{-7}
VEK280				
Floating	1.10	16.72	1.36	4.05×10^{-7}
Fixed	1.28	56.59	1.00	3.87×10^{-7}
Scalar	1.11	29.13	1.40	1.21×10^{-7}
Transfer	1.27	15.40	1.23	3.93×10^{-8}

C. Memory Errors

The final error type is a memory error that occurs when a single event upset (SEU) flips a bit in the program or data memory. As described earlier, the event registers are configured to count memory upsets whenever they are detected by the ECC circuitry. To make sure that the memory upsets are caught, the AI engines are configured to scrub the memory regularly during the spill. The memory scrub rate is configured by choosing the number of clock cycles between each scrub interval, which processes 16 bytes of a memory. The number of clock cycles for a desired scrub rate is calculated using Equation 3,

$$N_{CLKs} = \frac{flt}{ns} \quad (3)$$

where f is the frequency of the clock, l is the number of bytes processed at each scrub interval, t is the duration of radiation spills, n is the desired number of times the memory is scrubbed each spill, and s is the size of the memory being scrubbed in bytes. In our testing configuration, the number of clock cycles was set to scrub the AI program memory at least four times per spill. The ECC protected data memory used the same number of clock cycles as the program memory. In the ML tiles, the ECC protected memory has the same size as the program memory, so it was also scrubbed at least four times per spill. In the AIE tiles, the ECC protected memory is half the size as the program memory, so it was scrubbed at least eight times per spill.

The following equation is used to compute the cross-section of memory errors,

$$\sigma_{mem} = \frac{\sum_{i=1}^S m_i}{b \cdot \Phi_{total}}, \quad (4)$$

where m_i is the number of memory errors counted during spill i and b is the number of bits in the memory being scrubbed¹.

The memory test was updated for the September test by moving the memory write outside of the ECC-protected region, essentially leaving this memory static during a test. This test was performed on the VEK280 during the September test and its results are shown in Figure 10. The programmable logic configuration RAM (CRAM) upset data was also collected at this test and its cross-section is included with the memories for comparison.

D. APU0 Halting Errors

Although the reliability of the APU was not the focus of this paper, the proper operation of the APU was critical to the success of the AI engine tests. If APU0 halts, no communication can occur with the AI engines, and the run must be restarted. The halting of the APU0 was usually the reason ending a run. The APU0 halts were logged

¹The program memory of both engines is 16 KiB. The AI engines have two 4 KiB banks of ECC-protected memory each and the ML engines have two 8 KiB banks of ECC-protected memory each.

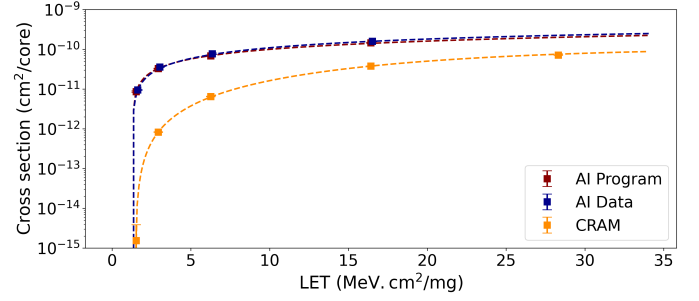


Fig. 10. AI Memory and CRAM SEU Cross-Section.

during the test enabling us to calculate the cross-section of APU0 halts. Unlike the AI engines in which multiple engines are being tested in parallel, the APU0 is a single core. This simplifies the cross-section calculation since there is no need to adjust for the number of active cores. The cross-section for the APU0 is calculated simply by dividing the total number of APU0 halts by the total fluence across all runs at a given LET, or $\sigma_{APU0} = e_{APU0} / \Phi$.

The APU0 halt cross-section is shown in Figure 11 and the Weibull fit parameters are reported in Table IV. These results are similar to the heavy ion results for the Low Power Domain and Full Power Domain results reported in [14]. It is interesting to note that the saturation cross-section of the APU0 halts is about two orders of magnitude higher than the saturation freeze cross-section of a single AI engine.

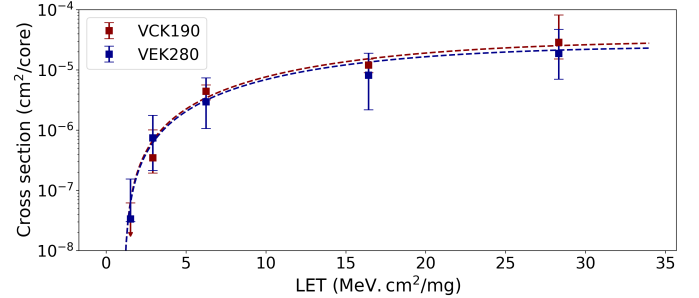


Fig. 11. APU SEFI Cross-Section.

E. Combined Cross-Section Comparison

Many errors were captured during these tests and the cross-sections of a variety of architectural components were calculated. It is helpful to compare the cross-section of these different components to understand the relative susceptibility of each component to radiation. Figure 12 shows the cross-section of the APU0 halts, PMC SEFIs, AIE SDC errors, and AIE freeze errors. The corresponding Weibull fit parameters for all these curves are reported in Table IV.

The most sensitive component in these tests is the APU0. For most systems using the AI engines, the APU0 is a critical component that must be operational for the

AI engines to function, so the APU0 halt cross-section is an important metric to consider when evaluating AI engine reliability. The next most sensitive component is the PMC, which has a saturation cross-section about three times lower than the APU0. The PMC is also essential for device operation as it handles the device scrubbing and many essential device management functions. Individual AI engines have a much lower saturation cross-section than the APU0 and PMC, with the AI SDC errors having a saturation cross-section about two orders of magnitude lower than the APU0 and the freeze errors having a saturation cross-section about 500 times lower than the APU0.

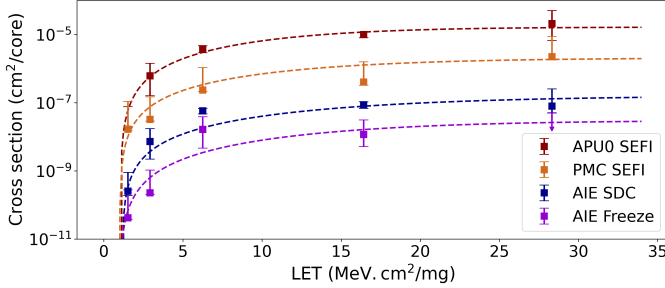


Fig. 12. Cross-Section Comparison of APU, PMC, AIE, and Memory Errors.

The results shown in Figure 12 can be used to estimate the overall reliability of a system using the Versal AI engines. The AI freeze or SDC error cross-section of a more complex system can be estimated by multiplying the per AI engine cross-section by the number of AI engines in the system, and the APU0 and PMC cross-sections can be used directly. These can be combined to estimate the overall system failure cross-section.

TABLE IV
WEIBULL FIT PARAMETERS FOR APU, PMC, AIE, AND MEMORY ERRORS.

Event	LET_0	W	s	σ
APU0 SEFI	1.04	13.40	1.70	1.65×10^{-5}
PMC SEFI	1.00	15.74	1.56	2.05×10^{-6}
AIE SDC	1.15	19.63	1.57	1.61×10^{-7}
AIE Freeze	1.01	17.76	1.73	3.00×10^{-8}
AIE Data Mem (bit)	1.29	43.31	0.80	4.54×10^{-10}
CRAM (bit)	1.46	24.41	1.73	1.10×10^{-10}

VIII. CONCLUSION

This work presents the heavy-ion testing approach for the AI/ML engines on the Versal Core and Edge series devices by simultaneously characterizing hundreds of processing elements within a single device. By leveraging the inherent spill structure of the NSRL beam and synchronizing device execution and data extraction with this structure, this work demonstrates a testing approach in which each beam spill is treated as a discrete experimental

interval. This enables temporal and spatial resolution of processing element failure that is not possible with traditional testing approaches.

The experimental results demonstrate that individual AI engines exhibit relatively low susceptibility to SDCs and freezes compared to system-critical components such as the APU and PMC, whose cross-sections are orders of magnitude higher. However, when hundreds of AI engines are operated concurrently, the aggregate system-level failure probability becomes non-negligible.

More broadly, this work introduces an experimental approach for radiation testing of highly parallel systems. By aligning data acquisition with beam macrostructure, it becomes possible to treat radiation exposure as a sequence of discrete, repeatable trials, enabling improved statistical interpretation of SEE behavior and facilitating correlation of faults with specific exposure intervals. This approach is particularly well-suited for emerging heterogeneous architectures that integrate large arrays of processing elements, where traditional steady-state or aggregate testing methods obscure important spatial and temporal effects.

REFERENCES

- [1] G. Furano, G. Meoni, A. Dunne, D. Moloney, V. Ferlet-Cavrois, A. Tavoularis, J. Byrne, L. Buckley, M. Psarakis, K.-O. Voss, and L. Fanucci, "Towards the use of artificial intelligence on the edge in space systems: Challenges and opportunities," *IEEE Aerospace and Electronic Systems Magazine*, vol. 35, no. 12, pp. 44–56, December 2020.
- [2] M. C. Casey, J. S. Goodwill, E. J. Wyrwas, R. A. Austin, C. M. Wilson, S. D. Stansberry, N. Gorius, and S. Aslam, "Single-event effects on commercial-off-the-shelf edge-processing artificial intelligence ASICs," *IEEE Transactions on Nuclear Science*, vol. 70, no. 8, pp. 1716–1723, August 2023.
- [3] S. Esposito, C. Albanese, M. Alderighi, F. Casini, L. Giganti, M. L. Esposti, C. Monteleone, and M. Violante, "COTS-based high-performance computing for space applications," *IEEE Transactions on Nuclear Science*, vol. 62, no. 6, pp. 2687–2694, December 2015.
- [4] M. Pignol, "COTS-based applications in space avionics," in *Design, Automation & Test in Europe Conference & Exhibition (DATE 2010)*, March 2010, pp. 1213–1219.
- [5] "Versal adaptive SoC AI engine architecture manual (AM009)," AMD, Tech. Rep., February 2026, Version 1.4.
- [6] P. Rech, "Artificial neural networks for space and safety-critical applications: Reliability issues and potential solutions," *IEEE Transactions on Nuclear Science*, vol. 71, no. 4, pp. 377–404, April 2024.
- [7] F. Libano, B. Wilson, M. Wirthlin, P. Rech, and J. Brunhaver, "Understanding the impact of quantization, accuracy, and radiation on the reliability of convolutional neural networks on FPGAs," *IEEE Transactions on Nuclear Science*, vol. 67, no. 7, pp. 1478–1484, July 2020.
- [8] L. Buckley, A. Dunne, G. Furano, and M. Tali, "Radiation test and in orbit performance of MpSoC AI accelerator," in *2022 IEEE Aerospace Conference (AERO)*, March 2022, pp. 199–207.
- [9] H.-B. Wang, Y.-S. Wang, J.-H. Xiao, S.-L. Wang, and T.-J. Liang, "Impact of single-event upsets on convolutional neural networks in Xilinx Zynq FPGAs," *IEEE Transactions on Nuclear Science*, vol. 68, no. 4, pp. 394–401, April 2021.
- [10] F. F. d. Santos, P. F. Pimenta, C. Lunardi, L. Draghetti, L. Carro, D. Kaeli, and P. Rech, "Analyzing and increasing the reliability of convolutional neural networks on GPUs," *IEEE Transactions on Reliability*, vol. 68, no. 2, pp. 663–677, June 2019.

- [11] B. L. Coelho, F. F. dos Santos, M. Saveriano, G. Allen, A. Daniel, S. Guertin, S. Vartanian, E. Wyrwas, C. Frost, and P. Rech, "Impact of radiation-induced effects on embedded GPUs executing large machine learning models," *IEEE Transactions on Nuclear Science*, vol. 72, no. 8, pp. 2652–2661, August 2025.
- [12] J. Gava, A. Sherjil, L. H. Laurini, E. Atukpor, R. P. Bastos, F. Moraes, R. Reis, and L. Ost, "Radiation-induced soft error assessment of a multithreaded mobilenet CNN model running in a multicore edge processor," *IEEE Transactions on Nuclear Science*, vol. 72, no. 8, pp. 2821–2829, August 2025.
- [13] R. L. R. Junior and P. Rech, "Reliability of Google's tensor processing units for convolutional neural networks," in *2022 52nd Annual IEEE/IFIP International Conference on Dependable Systems and Networks - Supplemental Volume (DSN-S)*, July 2022, pp. 25–27.
- [14] A. Dufour, F. Manni, F. Pierron, D. Dangla, F. Bezerra, J. Mekki, and P. Maillard, "Heavy-ion and proton single event effect (SEE) characterization of 7nm FinFET AMD Versal," in *2023 23rd European Conference on Radiation and Its Effects on Components and Systems*, September 2023, pp. 95–102.
- [15] P. Maillard, Y. P. Chen, J. Arver, V. Merugu, A. Shui, and M. Voogel, "Neutron and 64 MeV proton characterization of Xilinx 7nm Versal multicore scalar processing system (PS)," in *Radiation Effects Data Workshop*, July 2022, pp. 18–22.
- [16] H. Quinn, C. Corley, and P. Thelen, "Neutron-induced SEEs in the Xilinx Versal Prime," in *Radiation Effects Data Workshop (REDW)*, July 2022, pp. 1–8.
- [17] P. Maillard, A. Dhavle, Y. P. Chen, N. Fraser, Y. Chen, N. Vacirca, J. Barton, M. L. Voogel, and M. Sawant, "Protons evaluation of 7nm Versal AI engine (AIE) based radiation tolerant platform for deep learning applications," in *2024 IEEE Radiation Effects Data Workshop (REDW)*, July 2024, pp. 89–96.
- [18] H. Quinn, "Challenges in testing complex systems," *IEEE Transactions on Nuclear Science*, vol. 61, no. 2, pp. 766–786, April 2014.
- [19] S. M. Guertin, R. Some, P. Nsengiyumva, E. H. Cannon, M. Cabanas-Holmen, and J. Ballast, "Radiation specification and testing of heterogenous microprocessor SOCs," in *2019 19th European Conference on Radiation and Its Effects on Components and Systems (RADECS)*, April 2019, pp. 219–225.
- [20] I. Rodriguez-Ferrandez, L. Kosmidis, M. Tali, D. Steenari, A. Hands, and C. Bélanger-Champagne, "Proton evaluation of single event effects in the NVIDIA GPU Orin SoM: Understanding radiation vulnerabilities beyond the SoC," in *2024 IEEE 30th International Symposium on On-Line Testing and Robust System Design (IOLTS)*, July 2024, p. 149–155.
- [21] S. E. Michalak, A. J. DuBois, C. B. Storlie, H. M. Quinn, W. N. Rust, D. H. DuBois, D. G. Modl, A. Manuzzato, and S. P. Blanchard, "Neutron beam testing of high performance computing hardware," in *Radiation Effects Data Workshop (REDW)*, July 2011, pp. 169–176.
- [22] J. Goeders, W. Smith, M. Kastriotou, and M. Wirthlin, "A parallelized neutron radiation testing technique to understand failures within a complex SoC," *IEEE Transactions on Nuclear Science*, vol. 72, no. 4, pp. 1059–1067, April 2025.
- [23] B. Wie, M. K. Plante, A. Berkley, S. M. Guertin, and R. J. Nejad, "Static, dynamic, and application-level SEE results for a 49-core RHBD processor," in *2013 IEEE Radiation Effects Data Workshop (REDW)*, July 2013, pp. 77–84.
- [24] V. Vargas, P. Ramos, V. Ray, C. Jalier, R. Stevens, B. Dupont De Dinechin, M. Baylac, F. Villa, S. Rey, N.-E. Zergainoh, J.-F. Méhaut, and R. Velazco, "Radiation experiments on a 28 nm single-chip many-core processor and SEU error-rate prediction," *IEEE Transactions on Nuclear Science*, vol. 64, no. 1, pp. 483–490, January 2017.
- [25] "AI Engine Kernel Coding Best Practices Guide (UG1079)," AMD, Tech. Rep., October 2022, Version 2022.2.
- [26] C. La Tessa, M. Sivertz, I.-H. Chiang, D. Lowenstein, and A. Rusek, "Overview of the NASA Space Radiation Laboratory," *Life Sciences in Space Research*, vol. 11, pp. 18–23, November 2016.
- [27] S. Buchner, N. Kanyogoro, D. McMorrow, C. C. Foster, P. M. O'Neill, and K. V. Nguyen, "Variable depth Bragg peak method for single event effects testing," *IEEE Transactions on Nuclear Science*, vol. 58, no. 6, pp. 2976–2982, December 2011.
- [28] B. Wie, M. K. Plante, A. Berkley, S. M. Guertin, and R. J. Nejad, "Static, dynamic, and application-level see results for a 49-core RHBD processor," in *2013 IEEE Radiation Effects Data Workshop (REDW)*, November 2013, pp. 77–84.
- [29] C. Price, "Heavy ion testing of Versal ACAP AI engines," Master's thesis, Brigham Young University, April 2025.